





SVEUČILIŠTE U ZAGREBU
Fakultet elektrotehnike i računarstva

PARALELNO PROGRAMIRANJE

Poglavlje 03 - PRAM



PRAM MODEL

- PRAM – **IDEALNO paralelno računalo** za **IDEJNI razvoj** paralelnih algoritama (gornja granica performansi)
- **Svojstva:**
  - **MIMD** računalo sa **zajedničkom memorijom**
    - nije potrebna razmjena poruka
  - **proizvoljni** ili **zadani** broj procesora (p)
  - **sinkronizam:** svi procesori jednako brzo izvode svaku instrukciju
    - **lock-step** (nema potrebe za sinkronizacijom)
  - **algoritmi:** ključna riječ '***paralelno***' označava blok paralelnih naredbi
    - izvodi se na *potrebnom* broju procesora
    - **nije moguće ugniježđivanje**

PRAM MODEL

- Primjer PRAM programa:

```
…

A=42

PARALELNO (i=0 do n-1)
    niz[i]+=5;
    niz[i]*=2:
```

izvodi se slijedno, na jednom (glavnom) procesoru

- *svaki procesor radi sa svojim (i-tim) elementom niza, u lock-step načinu rada*
- *indeksi procesora nisu varijable u dijeljenoj memoriji, nego su dio upravljačke strukture*

FOR PETLJA

FOR (d = 0; d < log n; d++) {

SLIJEDNO: samo 1 procesor inkrementira *d*; *d* je dio upravljačkog toka, a ne varijabla u dijeljenoj memoriji

```
PARALELNO (i = 0 to n-1,  korak ...) {
        A[...] = ...
}
```

}

- **Interpretacija:**
  - svaki *d* → **jedna globalna faza**
  - timeline:

    d = 0 ⟶ svi procesori istovremeno rade jedan korak petlje
    d = 1 ⟶ svi procesori istovremeno rade jedan korak petlje
    d = 2 ⟶ svi procesori istovremeno rade jedan korak petlje

PRAM INAČICE

- Pristup memoriji:
  - istodobno samo jedan procesor (exclusive) - E
  - istodobno pristupa više (concurrent) - C
- Inačice:
  - **EREW** → najrestriktivnija*
  - **CREW**
  - (ERCW)
  - **CRCW** → dogovor o rezultatu za istovremeno pisanje:
    - **prioritetno** - uspijeva procesor s najmanjim indeksom
    - **operacijski** - nad svim podacima provodi se zadana operacija (npr. +) i upisuje rezultat
    - **slučajno** - slučajni procesor uspijeva (nedeterministički) - *default*

**ako radi za EREW, radit će i za ostale inačice (brže?); obrnuto ne vrijedi*

PRIMJERI ALGORITAMA

- Sortiranje, matrične operacije, grafovi
- Kombinatorički optimizacijski problem…

ALGORITAM ZBROJA PREFIKSA

- Default: EREW PRAM
- *All-prefix-sums, scan (inclusive scan)*
- Vrijedi za bilo koji **asocijativni binarni** operator

$$[a_0,\ a_1,\ \ldots\ ,\ a_{n-1}]$$

$$\downarrow \text{scan}$$

$$[a_0,\ (a_0 \oplus a_1),\ \ldots\ ,\ (a_0 \oplus a_1 \oplus \ldots \oplus a_{n-1})]$$

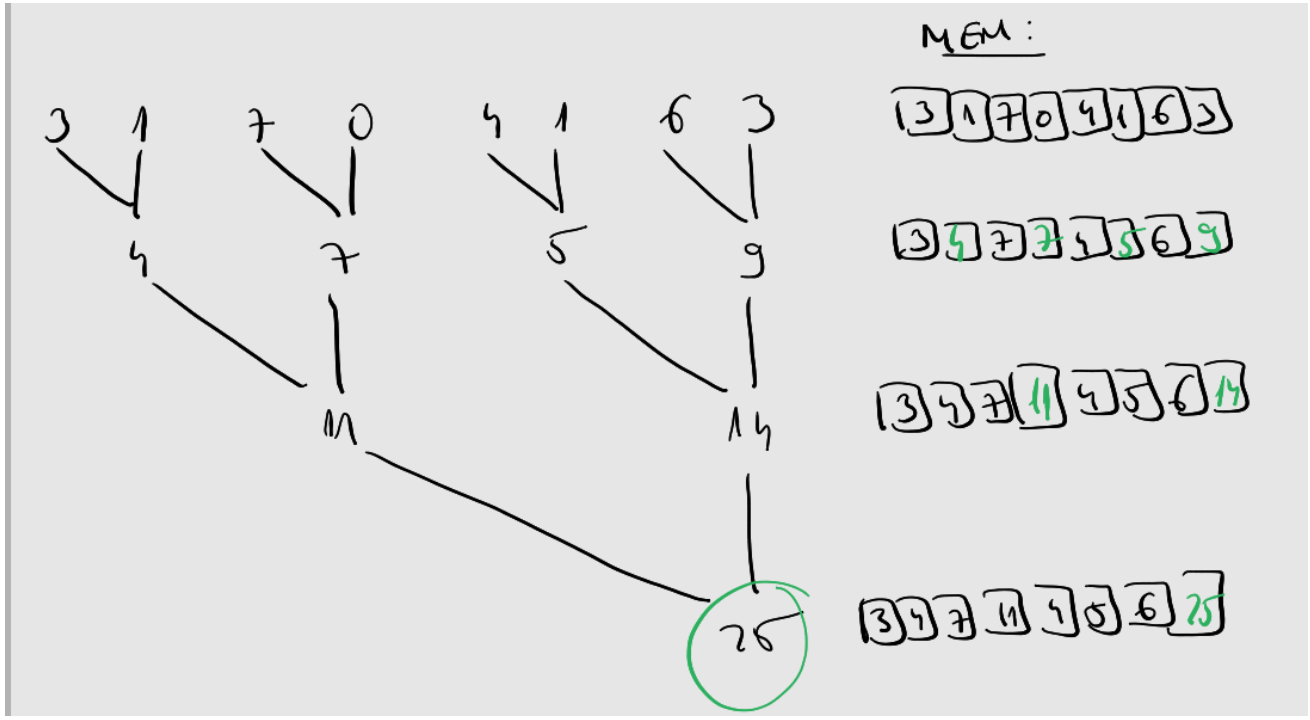
- Primjena: usporedba niza znakova, sortiranje, regex…
- Za ostvarenje potrebe pomoćne operacije: **reduce** i **prescan**

REDUCIRANJE

- Primjena zadanog operatora na sve elemente niza
  - npr. **+_Reduce**

    → rezultat: $(a_0 + a_1 + \ldots + a_{n-1})$

    → složenost: O(log n), (*default log: baza 2*)



REDUCIRANJE

```
algoritam +_reduciranje(A[], n)

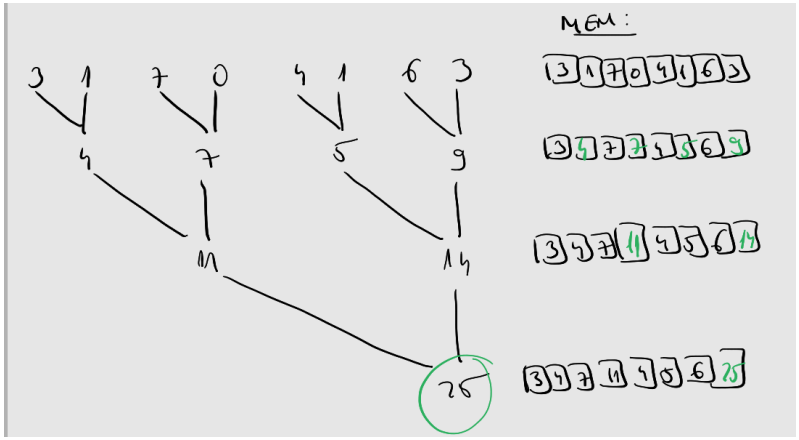
FOR d = 0 do (log n)-1

        PARALELNO (i od 0 do n-1; korak 2^(d+1) )

                A[i+2^(d+1)-1] += A[i+2^(d)-1];

        KRAJ_PARALELNO

KRAJ_FOR
```



PRESCAN

- *exclusive scan*

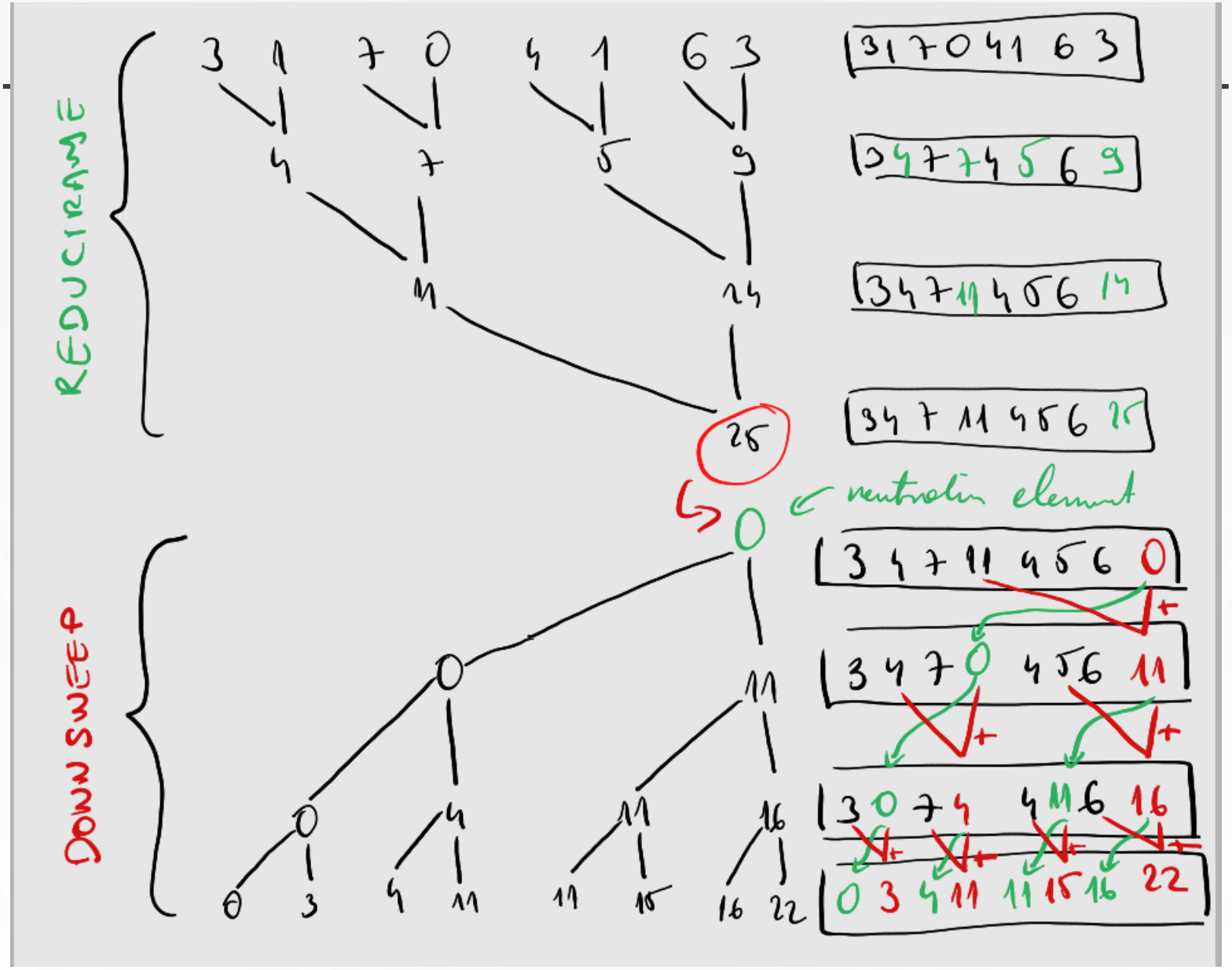
$$[a_0,\ a_1,\ \ldots\ ,\ a_{n-1}]$$

$$\downarrow \quad \text{prescan}$$

$$[I,\ a_0,\ (a_0 \oplus a_1),\ \ldots\ ,\ (a_0 \oplus a_1 \oplus \ldots \oplus a_{n-2})]$$

neutralni element, npr. '0' za zbrajanje

- nakon reduciranja (*upsweep*), gradi se novo stablo (*downsweep):*
  1. u **korijen** stabla upisuje se **neutralni** (ili zadani) element
  2. svaki čvor:
     - u **desni podlist** upisuje zbroj vlastite i vrijednosti lijevog podlista
     - u **lijevi podlist** upisuje svoju vrijednost


REDUCIRANJE
DOWN SWEEP
nevtralen element

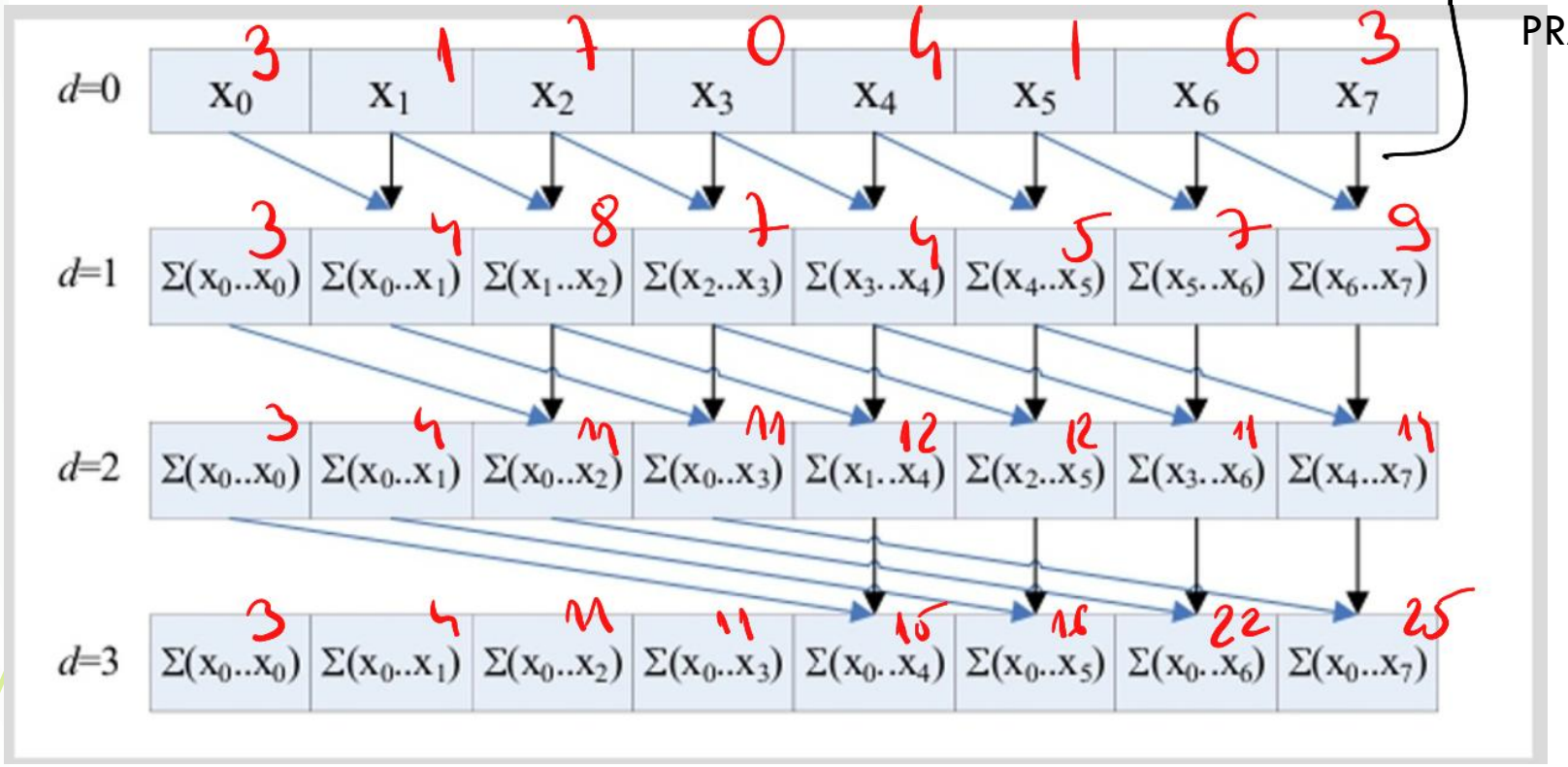
PRESCAN

```
algoritam down_sweep(A[], n)
A[n-1] = 0;          // neutralni elm.
FOR d = (log n)-1 do 0 korak -1
   PARALELNO (i od 0 do n-1; korak 2^(d+1) )
         temp[i] = A[i+2^(d)-1];      // spremi vrijednost lijevoga
         A[i+2^(d)-1] = A[i+2^(d+1)-1]; // postavi lijevog
         A[i+2^(d+1)-1] = A[i+2^(d+1)-1] + temp[i]; // postavi desnog (isto
                                                    mjesto u memoriji!)
   KRAJ_PARALELNO
KRAJ_FOR
```

- SCAN:
  - rezultat PRESCAN-a pomaknuti ulijevo
  - na kraj dodati (međurezultat) REDUCE

2 log(n) koraka, O(log(n))

CREW PRAM SCAN



ovaj korak nije moguć na EREW PRAM-u

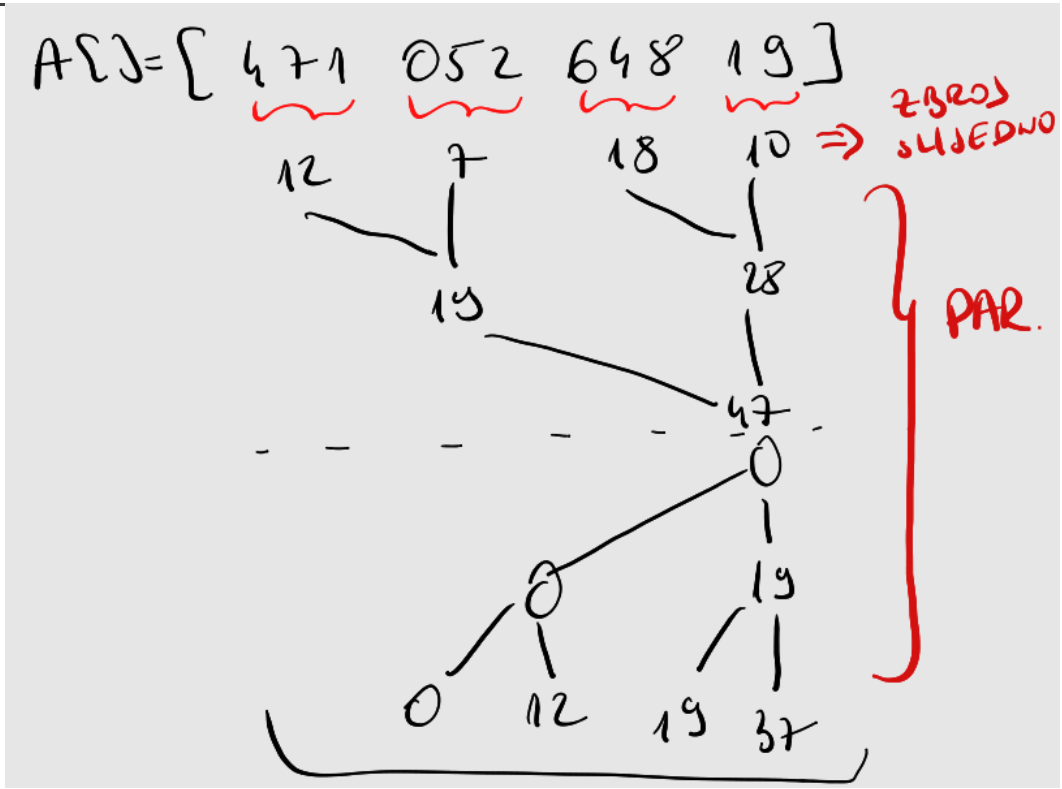
log(n)

PRESCAN ZA $p < {}^{n}/_{2}$

- svaki processor dobije ***n/p*** elemenata te ih SLIJEDNO obradi (npr. zbroji)
- zatim se nastavlja PARALELNO za ***p*** elemenata
- na kraju ponovno SLIJEDNO
- npr. n=11, p=4
  - P0, P1, P2 – 3 el.
  - P3 – 2 el.

$$O(n) = 2 \cdot \frac{n}{p} + 2 \cdot \log(n)$$

SLIJEDNO PARALELNO



POČ. EL. NIZA ⇒ P0: 0 | P1: 12 | P2: 19 | P3: 37
ZA SLIJEDNI PRESCAN ⇒ [4,7,1] [0,5,2] [6,4,8] [1,9]
(NA SVAKOM PROCESORU) [0,4,11] [12,12,17] [19,25,27] [37,38]

PRESCAN

- ako je *p* različit od 2^x, onda DODAMO NULE na kraj niza da "uposlimo" preostale procesore
- PRIMJENA:
  - najveći/najmanji element u nizu → MIN/MAX REDUCE
  - alokacija resursa → +_SCAN (prescan)
  - provjera uređenosti niza → dodatno polje R[], AND/OR REDUCE
  - sortiranje, npr. RADIX sort → operacija SPLIT…

ZADATAK 1

Napisati algoritam za CRCW/CREW/EREW PRAM računalo koji će za zadano polje P[] provjeriti jesu li **svi elementi polja isti** (jednakih vrijednosti).

Za polje od n elemenata na raspolaganju je n procesora. Rezultat mora biti zapisan u jednoj izlaznoj varijabli.

Ocijeniti složenost algoritma.

ZADATAK 2 – ALOKACIJA RESURSA

A Inclusive Scan Application Example

- Assume that we have a 100-inch sausage to feed 10
- We know how much each person wants in inches
  - [3 5 2 7 28 4 3 0 8 1]
- How do we cut the sausage quickly?
- How much will be left

- Method 1: cut the sections sequentially: 3 inches first, 5 inches second, 2 inches third, etc.
- Method 2: ?

RADIX SORT

- nužno definirati **BROJ BITOVA** (*znamenki*) u zapisu
- kreće od znamenke **najmanje težine ➔ operacija SPLIT**
  - na osnovi promatranog bita. sve brojeve s "0" ispred brojeva s "1" u promatranom bitu
- očuvan relativni poredak unutar iste kategorije
- složenost (binarni zapis):

  O(n log n)

PRIMJER (3-bitni zapis):

```
   A = [ 5 7 3 1 4 2 7 2 ]
bits = [ 1 1 1 1 0 0 1 0 ]   LSB
-------------------------------
   A = [ 4 2 2 5 7 3 1 7 ]
bits = [ 0 1 1 0 1 1 0 1 ]
-------------------------------
   A = [ 4 5 1 2 2 7 3 7 ]
bits = [ 1 1 0 0 0 1 0 1 ]   MSB
-------------------------------
   A = [ 1 2 2 3 4 5 7 7 ]
```

RADIX SORT - SPLIT

- paralelizira se operacija SPLIT:

O(n/p + log p)

```
   A = [ 5 7 3 1 4 2 7 2 ]
bits = [ 1 1 1 1 0 0 1 0 ]
```

LSB

```
DOLJE = [ 0 0 0 0 0 1 2 2 ]
```

+_prescan (!bits[])

```
 POM = [ 4 3 2 1 1 1 0 0 ]
```

+_prescan_unatrag (bits[])

```
GORE = [ 3 4 5 6 6 6 7 7 ]
```

7 – POM[])

```
INDEX = [ 3 4 5 6 0 1 7 2 ]
```

AKO (bits[i]==1) GORE[i]
INAČE DOLJE[i]

```
   A = [ 4 2 2 5 7 3 1 7 ]
```

elementi iz A[] se razmještaju na mjesto koje piše u INDEX

ZADATAK 3

Napisati algoritam za CRCW/EREW PRAM računalo koji će za zadano polje P[] provjeriti ima li u polju elemenata **jednakih vrijednosti (duplikata).**

Za polje od n elemenata na raspolaganju je n procesora. Rezultat mora biti zapisan u jednoj izlaznoj varijabli.

Ocijeniti složenost algoritma.

ZADATAK 4

U nepoznatom elementu niza A[] duljine n zapisana je **cjelobrojna vrijednost različita od nule**, dok je vrijednost svih ostalih elemenata nula.

Napisati algoritam za CRCW/EREW PRAM koji će vrijednost iz nepoznatog elementa prepisati u sve elemente niza.

Ocijeniti složenost algoritma.

ZADATAK 5

Napisati algoritam za CRCW/EREW PRAM računalo koji će za zadano polje P[] odrediti **broj različitih vrijednosti elemenata polja**. Npr. za polje [1, 2, 1, 3, 4, 2, 5, 1] rezultat iznosi 5.

Za polje od n elemenata na raspolaganju je n procesora. Rezultat mora biti zapisan u jednoj izlaznoj varijabli.

Ocijeniti složenost algoritma.

ZADATAK 6

Napisati algoritam za EREW PRAM računalo složenosti manje od n koji će za dva ulazna niza znakova A[] i B[] duljine n odrediti koji je **prvi po abecedi** (program treba ispisati "A", "B" ili "jednaki").

Na raspolaganju je funkcija reduciranja koja provodi proizvoljnu binarnu operaciju nad elementima polja.

Ocijeniti složenost algoritma. Proizvoljnu operaciju možete definirati programski.